Methodology & Contributions

Our Chosen Development Method – Rational Unified Process

For this project, we have chosen to follow the Rational Unified Process (RUP) methodology due to its iterative and flexible approach to development. Unlike more rigid models, RUP allows requirements to evolve over time, making it better suited to projects where changes may occur during development.

One of the key advantages of RUP is its support for continuous collaboration within the team. Regular iterations allow us to stay aligned on progress, and tasks can be redistributed if a team member encounters difficulties, ensuring that the overall workflow remains efficient.

In comparison, the Waterfall model follows a linear structure with fixed requirements, which limits the ability to make changes once development has begun. While this approach can be effective in some cases, it lacks the adaptability required for our project.

Overall, we selected RUP as it provides greater flexibility, encourages ongoing communication, and enables us to respond more effectively to challenges throughout the development process.

RUP – General Overview

The Rational Unified Process is an iterative and incremental software development methodology designed to improve project organisation and adaptability.

RUP is structured into five key phases: Inception, Elaboration, Construction, Transition, and Production. Each phase focuses on a specific stage of development, allowing the project to progress in manageable increments rather than as a single continuous process.

In addition to these phases, RUP includes several disciplines such as Requirements, Analysis, and Design. These disciplines run throughout the project lifecycle and ensure that each aspect of development is consistently addressed.

The methodology is architecture-centric and supports component-based development, which promotes reusability and scalability. It is particularly effective in projects that require strong documentation and risk management.

However, RUP does have some limitations. It requires team members to have a solid understanding of their roles and responsibilities, and it relies heavily on effective risk management to succeed.

System Requirements and Features

The system requirements are divided into functional and non-functional categories, ensuring that both the behaviour of the system and its performance are clearly defined.

Functional Requirements

The system is designed to address the operational challenges of a multi-location apartment management company by providing a centralised and role-based solution.

Account and User Management

The system supports multiple user roles, each with specific permissions to ensure secure and appropriate access to data:

- **Front-desk staff** can register new tenants by capturing key details such as National Insurance number, name, contact information, occupation, references, apartment preferences, and lease duration. They are also responsible for handling tenant enquiries, complaints, and logging maintenance requests.
- **Finance managers** manage financial operations, including tracking payments, generating invoices, monitoring late payments, and producing financial reports.
- **Maintenance staff** handle maintenance requests by recording issues, updating their progress, and logging resolution details including time taken and costs.
- **Administrators** have full control over a specific location, including managing user accounts, apartment data, lease agreements, and generating reports.
- **Managers** oversee system-wide operations, monitor occupancy levels across locations, generate performance reports, and support business expansion into new cities. They also have high-level access to all dashboards.

Tenant Management

The system maintains comprehensive tenant records, allowing users to add, update, and remove tenant information. It also tracks lease agreements, payment history, and complaint logs.

The system supports early lease termination requests, enforcing business rules such as a required one-month notice period and a penalty fee equivalent to 5% of the monthly rent.

Apartment Management

The apartment management component enables the registration and management of properties across multiple locations. Each apartment record includes attributes such as location, type, number of rooms, and monthly rent.

The system supports assigning apartments to tenants, allowing accurate tracking of occupancy levels. It also integrates with maintenance management to track the lifecycle of maintenance requests associated with each property.

Payment and Billing

The system manages billing processes by generating invoices, recording payments, and identifying late payments. It also produces payment summaries and due notices.

While the system does not process real transactions, it simulates billing operations by generating receipts and maintaining accurate financial records for reporting purposes.

Reporting

The reporting component provides real-time insights to support decision-making. Key reports include:

- Occupancy reports by apartment and location
- Financial summaries comparing collected and outstanding rent
- Lease agreement reports, showing trends in signed and expiring leases over time

---

Maintenance Management

The maintenance component manages all reported issues from initial request to resolution. Staff can log issues, assign priorities, schedule maintenance work, and communicate updates to tenants.

The system tracks the time required to resolve issues and associated costs, enabling better performance monitoring and cost control.

Non-Functional Requirements

The system is designed to meet key non-functional requirements that ensure reliability, security, and scalability.

- **Security:** Role-based access control ensures that sensitive tenant and financial data is only accessible to authorised users. Authentication mechanisms protect against unauthorised access.
- **Scalability:** The system supports multiple locations and can expand to include additional cities without performance degradation.
- **Efficiency:** A centralised database eliminates data duplication and ensures consistency across all locations, improving overall system performance.
- **Usability:** The system features an intuitive graphical user interface, reducing the need for extensive staff training and improving productivity.
- **Reliability:** The system ensures consistent data availability and supports real-time updates across all locations.

Contributions and Roles

Contributions to the project were distributed among team members based on individual strengths and areas of interest, ensuring an efficient and balanced workflow. The team followed the iterative nature of the Rational Unified Process, allowing members to collaborate, provide feedback, and support one another throughout different stages of development.

Each member was assigned primary responsibility for specific components of the system, while still contributing to shared tasks such as testing, debugging, and refining system requirements.

- **Aaron AB** was responsible for developing the Admin and Manager interfaces, as well as designing the overall file structure and base architecture of the application. He also

contributed to graph generation, created page components, and implemented features for exporting graphs and statistics.

- **Oliver C** focused on the Maintenance component, including both the user interface and backend design for handling maintenance requests. This involved structuring how issues are logged, tracked, and resolved within the system. He also contributed to the Manager component and the early foundations of the application.
- **Oliver M** developed the Front Desk functionality, particularly tenant management features, and was responsible for generating mock data used for testing and validating system behaviour.
- **Shamy A** worked on the Finance component and contributed to the overall user interface design. Additionally, he supported the development of modular page components, assisted in implementing graphical reporting features, and improved the UI design.
- **Nick G** contributed to the development of the Front Desk interface, supporting tenant-related functionality and ensuring consistency across the user experience.

Throughout the project, collaboration was maintained through regular communication and iteration reviews. This allowed the team to address challenges, redistribute tasks when necessary, and ensure that all components of the system integrated effectively.

Version Control and Collaboration

Version control for the project was managed using GitHub, allowing the team to collaborate efficiently and maintain a structured development workflow. Each team member worked on their assigned components using separate branches, which helped prevent conflicts and ensured that features could be developed independently before being integrated into the main codebase.

GitHub also provided a clear history of changes, enabling the team to track progress, review contributions, and revert to previous versions if necessary. This was particularly useful during debugging and when refining features across multiple iterations.

The use of version control supported the iterative nature of the Rational Unified Process by allowing continuous updates, testing, and integration of system components. It also improved overall team coordination, ensuring that all members were working with the most up-to-date version of the project.